

Python programming



MC103 PROJECT

SUBMITTED TO:

Prof. CP Singh
Ms Bhramvidya
Ms Priya Yadav
Ms Simran

SUBMITTED BY:

Parv Bansal

AI Resume Critiquer

THE SMART RESUME ANALYZER



PYTHON WITH AI

Presented By

Parv Bansal 25/A12/049

CERTIFICATE OF PROJECT COMPLETION

This is to certify that Parv Bansal (25/A12/049) of MCE-2 have successfully completed the MC103 Python Programming Project titled “AI Resume Critiquer – The Smart Resume Analyzer” under the guidance of Prof. C.P. Singh and the supervision of Ms. Bhramvidya, Ms. Priya Yadav, and Ms. Simran, at Delhi Technological University (DTU).

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to Prof. C. P. Singh, Ms. Bhramvidya, Ms. Priya Yadav, and Ms. Simran for their valuable guidance, encouragement, and constant support throughout the completion of our MC103 Python Programming Project titled

“AI Resume Critiquer – The Smart Resume Analyzer.”

Their insightful feedback and motivation helped us explore the practical aspects of Python programming and Artificial Intelligence, enhancing both our technical understanding and problem-solving abilities.

This experience has deepened our knowledge of Python’s applications in real-world scenarios and strengthened our confidence in developing AI-powered tools.

Parv Bansal (25/A12/049)

MCE–2, Delhi Technological University (DTU)

SOFTWARE USED

- Programming Language: Python 3.11
- IDE / Editor: Visual Studio Code (VS Code)
- Framework: Streamlit (for web app interface)
- Libraries Used: PyPDF2, OpenAI, python-dotenv, io, os
- Environment: Virtual Environment (.venv)
- Operating System: macOS

INTRODUCTION

The project “AI Resume Critiquer – The Smart Resume Analyzer” is an Artificial Intelligence–based application developed using Python. It is designed to analyze resumes and provide intelligent, personalized feedback to help users improve their resume quality. The system uses Natural Language Processing (NLP) through the OpenAI API to evaluate the content of a resume based on factors such as clarity, structure, skill presentation, and overall impact. Users can upload their resumes in PDF or text format through a Streamlit-based web interface, making the tool simple and interactive to use. Once uploaded, the system extracts text using the PyPDF2 library and then sends it to the AI model for detailed analysis and suggestions.

This project demonstrates the practical integration of AI and Python in real-world applications, combining file handling, text processing, and AI-driven evaluation. It aims to assist students and professionals in enhancing their resumes to align better with job roles, ultimately improving employability. Through this project, the team gained valuable insights into AI implementation, API integration, and web application development, showcasing the versatility and power of Python in building smart, user-focused solutions.

SOURCE CODE

```
import streamlit as st

import PyPDF2

import os

from dotenv import load_dotenv

from openai import OpenAI

# --- page setup (must be near the top) ---

st.set_page_config(page_title="AI Resume Critiquer", page_icon="📄",
layout="centered")

# --- env / api key ---

load_dotenv()

OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")

st.title("AI Resume Critiquer")

st.markdown("Upload your resume and get AI-powered feedback tailored to your target role.")

uploaded_file = st.file_uploader("Upload your resume (PDF or TXT)", type=["pdf", "txt"])

job_role = st.text_input("Enter the job role you're targeting (optional)")

analyze = st.button("Analyze Resume", type="primary")

def extract_text_from_pdf(file_like) -> str:

    """Extract text from a text-based PDF (not scanned)."""

    try:

        reader = PyPDF2.PdfReader(file_like)
```

```

    text = ""

    for page in reader.pages:

        try:

            text += (page.extract_text() or "") + "\n"

        except Exception:

            # if a page fails to extract, skip it gracefully

            pass

    return text

except Exception as e:

    raise RuntimeError(f"Could not read PDF: {e}")

def extract_text(uploaded_file) -> str:

    """Return decoded text from uploaded PDF or TXT."""

    if uploaded_file.type == "application/pdf":

        return extract_text_from_pdf(uploaded_file)

    # TXT file

    content = uploaded_file.read()

    try:

        return content.decode("utf-8")

    except UnicodeDecodeError:

        return content.decode("latin-1", errors="ignore")

def make_prompt(resume_text: str, role: str) -> str:

    target = role.strip() if role else "general job applications"

    return f"""

You are an expert resume reviewer with years of experience in HR and recruitment.

```

Please analyze this resume and provide constructive, specific feedback.

Focus on:

- 1) Content clarity and impact
- 2) Skills presentation (relevance, scannability, quantification)
- 3) Experience bullets (action verbs, metrics, outcomes, ordering)
- 4) ATS-friendliness (sections, keywords, formatting risks)
- 5) Specific improvements for {target}
- 6) End with a short prioritized checklist

Resume:

```
\{"\"{resume_text}\"\"\"
```

Use clear headings and concise bullet points. If the resume seems too short/long, say so and suggest fixes.

"""

```
if analyze:
```

```
    # 1) sanity checks
```

```
    if not OPENAI_API_KEY:
```

```
        st.error("OPENAI_API_KEY not found. Add it to your `.env` file in this folder.")
```

```
        st.stop()
```

```
    if not uploaded_file:
```

```
        st.error("Please upload a PDF or TXT file.")
```

```
        st.stop()
```

```
    # 2) extract text
```

```
    try:
```

```
        resume_text = extract_text(uploaded_file)
```

```

except Exception as e:

    st.error(f"Error reading file: {e}")

    st.stop()

if not resume_text.strip():

    st.error("No text found. If your PDF is scanned, export a *Searchable PDF* or
paste the text into a .txt file.")

    st.stop()

# 3) call OpenAI

try:

    client = OpenAI(api_key=OPENAI_API_KEY)

    prompt = make_prompt(resume_text, job_role)

    resp = client.chat.completions.create(

        model="gpt-4o-mini",

        messages=[

            {"role": "system", "content": "You are a precise, constructive, senior
HR resume reviewer."},

            {"role": "user", "content": prompt},

        ],

        temperature=0.4,

        max_tokens=900,    )

    output = resp.choices[0].message.content

    st.subheader("Analysis Results")

    st.markdown(output)

except Exception as e:

    st.error(f"OpenAI request failed: {e}")

```

OUTPUT INTERFACE

You can now view your Streamlit app in your browser.

Local URL: <http://localhost:8506>

Network URL: <http://192.168.1.46:8506>

AI Resume Critiquer

Upload your resume and get AI-powered feedback tailored to your target role.

Upload your resume (PDF or TXT)



Drag and drop file here

Limit 200MB per file • PDF, TXT

Browse files

Enter the job role you're targeting (optional)

Analyze Resume

TRIAL RUN

AI Resume Critiquer

Upload your resume and get AI-powered feedback tailored to your target role.

Upload your resume (PDF or TXT)



Drag and drop file here

Limit 200MB per file • PDF, TXT

Browse files



Parv_Bansal_Resume.pdf 4.8KB



Enter the job role you're targeting (optional)

Software Engineer

Analyze Resume

Analysis Results

Resume Review for Parv Bansal

1) Content Clarity and Impact

- **Strengths:**
 - The resume clearly outlines educational background and relevant experiences.
 - The "About" section effectively conveys passion and goals in AI and technology.
- **Areas for Improvement:**
 - The language can be made more concise. Avoid redundancy (e.g., "contributing to AI research & development initiatives" could be simplified).
 - Consider a stronger opening statement that highlights key qualifications or achievements.

2) Skills Presentation

- **Strengths:**
 - Skills are relevant to a Software Engineer role and cover a range of important areas.
- **Areas for Improvement:**
 - Skills should be more scannable. Consider using bullet points or a table format for better visibility.
 - Quantify skills where possible (e.g., "Python: Developed X projects" or "Public Speaking: Delivered Y presentations").

3) Experience Descriptions

- **Strengths:**
 - The use of action verbs (e.g., "Contributing," "Leading," "Managing") is good.
- **Areas for Improvement:**
 - Use more specific metrics or outcomes to demonstrate impact (e.g., "Increased outreach by X% through branding initiatives").
 - Order experiences chronologically, starting with the most recent. This helps in showcasing the most relevant experience first.

4) ATS-Friendliness

- **Strengths:**
 - The resume has clear sections which are beneficial for ATS scanning.
- **Areas for Improvement:**
 - Ensure consistent formatting (e.g., use the same date format throughout).
 - Avoid using special characters (like " ") that may confuse ATS systems; replace with standard bullet points.
 - Include relevant keywords from job descriptions for software engineering roles to improve ATS ranking.

5) Specific Improvements for Software Engineer

- **Technical Projects:**
 - Include a section for technical projects or coursework relevant to software engineering. Highlight programming languages, frameworks, or tools used.
- **Internship Details:**
 - Expand on internship experiences with specific projects or technologies used. Focus on contributions that align with software engineering tasks.
- **Certifications:**
 - Consider adding a brief description of the skills gained from each certification to demonstrate practical application.

6) Final, Prioritized Checklist

- **Content Clarity:**
 - Simplify and strengthen the "About" section.
- **Skills Presentation:**
 - Use bullet points for skills and quantify where possible.
- **Experience Descriptions:**
 - Add metrics and outcomes to each role.
 - Organize experiences in reverse chronological order.
- **ATS-Friendliness:**
 - Standardize formatting and avoid special characters.
 - Include relevant keywords from job descriptions.
- **Technical Focus:**
 - Add a section for technical projects and elaborate on internship experiences.

By addressing these areas, Parv can enhance the effectiveness of the resume, making it more impactful and aligned with software engineering roles.

#CODE IS SUCCESSFUL